

Assignment 11: Supervised Machine Learning with Python

Name Ayleen Santander

```
#Import Necessary Libraries
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
```

```
#1.Dataset Selection:
#Load data Titanic
Tit = pd.read_csv('titanic (4).csv')

print(Tit.info())
print(Tit.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  -
 0   PassengerId    891 non-null    int64
 1   Survived       891 non-null    int64
 2   Pclass         891 non-null    int64
 3   Name           891 non-null    object
 4   Sex            891 non-null    object
 5   Age            714 non-null    float64
 6   SibSp          891 non-null    int64
 7   Parch          891 non-null    int64
 8   Ticket         891 non-null    object
 9   Fare           891 non-null    float64
10   Cabin          204 non-null    object
11   Embarked       889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            177
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          687
Embarked       2
dtype: int64
```

```
# 2. Clean Age (fill NaN with mean)
Tit['Age'] = Tit['Age'].fillna(round(Tit['Age'].mean())).astype(int)
```

```
# 3. Clean Embarked (fill NaN with mode)
Tit['Embarked'] = Tit['Embarked'].fillna(Tit['Embarked'].mode()[0])
```

```
Tit.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	1	22	1	0	21171	7.2500	8	2
1	2	1	1	Cumings, Mrs. John Bradley	0	38	1	0	17599	71.2833	2	0
2	3	1	3	Heikkinen, Miss. Laina	0	26	0	0	3101282	7.9250	8	2
3	4	1	1	Futrelle, Mrs. Jacques Heath	0	35	1	0	113803	53.1000	2	2
4	5	0	3	Allen, Mr. William Henry	1	35	0	0	373450	8.0500	8	2

Next steps: [Generate code with Tit](#) [New interactive sheet](#)

```
# 4. Clean Ticket (extract numeric part only)
Tit['Ticket'] = Tit['Ticket'].str.extract(r'(\d+)\$')
Tit['Ticket'] = pd.to_numeric(Tit['Ticket'], errors='coerce').fillna(0).astype(int)
```

```
# 5. Clean Cabin – extract deck letter (A, B, C...), fill missing with 'Unknown'
Tit['Cabin'] = Tit['Cabin'].str[0]
Tit['Cabin'] = Tit['Cabin'].fillna('Unknown')
```

```
# 6. Clean Name (remove text in parentheses, optional/cosmetic)
Tit['Name'] = Tit['Name'].str.replace(r'\(.*?\)', '', regex=True).str.strip()

print(f'After cleaning:\n', Tit.isnull().sum())
```

```
After cleaning:
 PassengerId    0
  Survived      0
   Pclass      0
    Name       0
    Sex       0
    Age       0
   SibSp      0
   Parch      0
  Ticket      0
   Fare      0
  Cabin       0
 Embarked     0
dtype: int64
```

```
# 7. Encode categorical columns
le_sex = LabelEncoder()
Tit['Sex'] = le_sex.fit_transform(Tit['Sex']) # male: 1, female: 0
```

```
le_embarked = LabelEncoder()
Tit['Embarked'] = le_embarked.fit_transform(Tit['Embarked']) # S=2, C=0, Q=1
```

```
le_cabin = LabelEncoder()
Tit['Cabin'] = le_cabin.fit_transform(Tit['Cabin']) # deck letters -> numbers
```

```
print(Tit.head())
```

	PassengerId	Survived	Pclass	Name	Sex	Age	\
0	1	0	3	Braund, Mr. Owen Harris	1	22	
1	2	1	1	Cumings, Mrs. John Bradley	0	38	
2	3	1	3	Heikkinen, Miss. Laina	0	26	
3	4	1	1	Futrelle, Mrs. Jacques Heath	0	35	
4	5	0	3	Allen, Mr. William Henry	1	35	

	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	21171	7.2500	8	2
1	1	0	17599	71.2833	2	0
2	0	0	3101282	7.9250	8	2
3	1	0	113803	53.1000	2	2
4	0	0	373450	8.0500	8	2

```
# 8. Define X and y – predicting Sex from Age, Fare, Cabin
X = Tit[['Age', 'Fare', 'Cabin']]
y = Tit['Sex']
```

```
# 9. Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

```
# 10. Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# 11. Model training
models = {
    'Logistic Regression': LogisticRegression(), 'Random Forest': RandomForestClassifier(), 'SVM': SVC()
}

results = {}

for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    accuracy = accuracy_score(y_test, y_pred)
    report = classification_report(y_test, y_pred)
    results[name] = (accuracy, report)

for model_name, (accuracy, report) in results.items():
    print(f"{model_name} Accuracy: {accuracy:.2f}")
    print(report)
```

```
Logistic Regression Accuracy: 0.66
      precision    recall  f1-score   support

     0       0.71      0.18      0.29         67
     1       0.66      0.96      0.78        112

   accuracy          0.66          179
  macro avg          0.68          179
 weighted avg          0.68          179

Random Forest Accuracy: 0.65
      precision    recall  f1-score   support

     0       0.53      0.54      0.53         67
     1       0.72      0.71      0.72        112

   accuracy          0.65          179
  macro avg          0.63          179
 weighted avg          0.65          179

SVM Accuracy: 0.66
      precision    recall  f1-score   support

     0       0.64      0.21      0.31         67
     1       0.66      0.93      0.77        112

   accuracy          0.66          179
  macro avg          0.65          179
 weighted avg          0.65          179
```

Model Evaluation and Final Analysis

Overall accuracy is similar across all three models (65-66%), but accuracy alone is misleading here because the classes are imbalanced (67 samples of class 0/female vs 112 of class 1/ male in the test set.)

- Logistic Regression class 0 (female) recall = 0.18/ Class 1(male) Recall =0.96
- Random Forest class 0(female) Recall = 0.54/ Class 1(male) Recall= 0.71
- SVM class 0(female)Recall = 0.21/class 1(male)Recall= 0.93
- Logistic Regression and SVM ~ 0.66 are basically just guessing "male" most of the time. They catch 93 -96% of actual males but only 18-21% of actual females. That's why accuracy looks okay(0.66), it's inflated by the majority class(male=112 samples), not because the model is actually good at distinguishing sexes. -Random Forest is the most balanced, recall of 0.54 and 0.71 across classes, even though its overall accuracy(0.65) is the lowest of the three it's not "cheating" by defaulting to the majority class as much.

Conclusion

While Logistic Regression and SVM report similar accuracy to Random Forest, their macro-average F1-scores (0.53 and 0.54) are lower, revealing they perform poorly on the minority class (female). Random Forest, despite marginally lower accuracy, shows more balanced performance (macro F1 = 0.63), suggesting Age, Fare, and Cabin alone are weak predictors of Sex, and the models are partially exploiting class imbalance rather than learning a genuine signal.